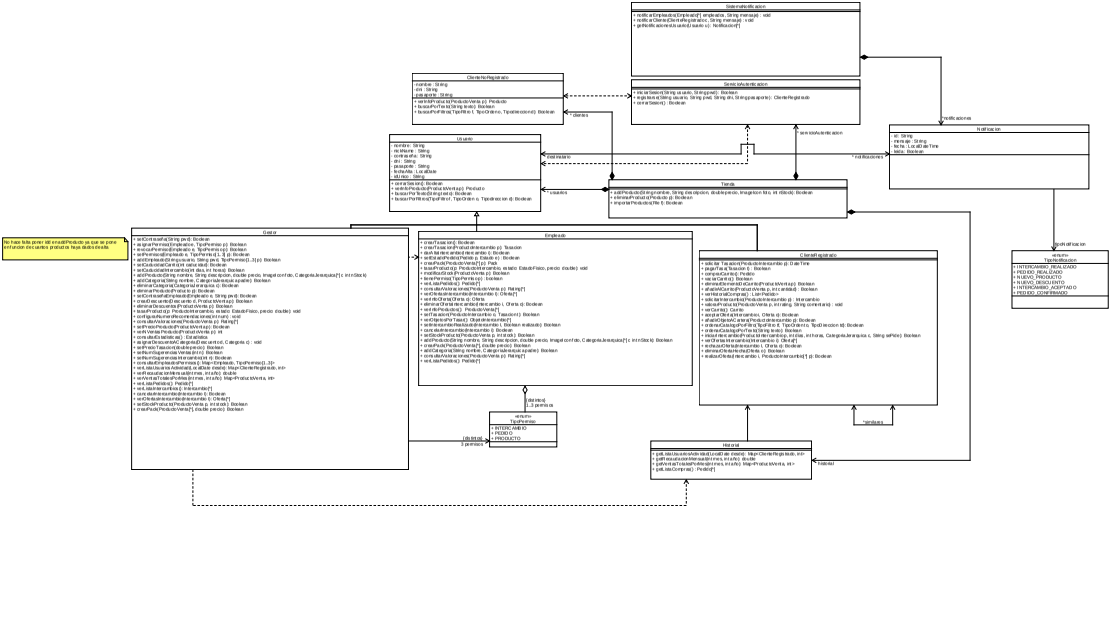
A Ellos

Documento de Diagramas

Jaime Jiménez Caballero
Luis Oro Arévalo
Diego Terrés Morata
Arturo Pérez Noves

Fecha: 26/02/2026


SistemaNotificacion
+ notificarEmpleado(Empleado[*] empleados, String mensaje) : void
+ getNotificacionesNoLeidas(int usuarioId, int[0..1] limit) : int
+ getNotificacionesNoLeidas(int usuarioId) : Notificacion[*]
ClienteRegistrado
- nombre : String
- email : String
- passwordHash : String
+ verificarVisualizaProductoVenta(p) : Producto
+ buscarPorTipo(String tipo) : Boolean
+ buscarPorRifNroTipoRifNro1, TipoCliente, TipoRifNro1rif) : Boolean
SesionAutenticacion
+ iniciarSesion(String usuario, String pass) : Boolean
+ registrarUsuario(int id, String pass, String rifs, String usuario) : ClienteRegistrado
+ cerrarSesion() : Boolean
Notificacion
- id : String
- mensaje : String
- fecha : LocalDateTime
- leida : Boolean
Usuario
- nombre : String
- apellidoPaterno : String
- apellidoMaterno : String
- rifNro : int
- passwordHash : String
- fechaAlta : LocalDateTime
- rifTipo : String
+ cerrarSesion() : Boolean
+ verificarVisualizaProductoVenta(p) : Producto
+ buscarPorTipo(String tipo) : Boolean
+ buscarPorRifNroTipoRifNro1, TipoCliente, TipoRifNro1rif) : Boolean
Venta
+ addProducto(String nombre, String descripcion, double precio, Integer cantidad) : Boolean
+ eliminarProducto(Producto p) : Boolean
+ importarProductosXRifNro() : Boolean
Gerente
+ setCentralesRif(String pv) : Boolean
+ crearTasaUnPrivatizaMonedaNro(int Nro) : Boolean
+ eliminarPermisoEmpleado(e, TipoPermiso p) : Boolean
+ setPermisoEmpleado(e, TipoPermiso p) : Boolean
+ setEstadoCuentaCliente(int cliente, int[0..1] estado) : Boolean
+ setEstadoCuentaClienteCambiar(int cliente, int[0..1] estado) : Boolean
+ setCodigoCuentaClienteCambiar(int cliente, int[0..1] codigo) : Boolean
+ addProducto(String nombre, String descripcion, double precio, Integer cantidad, CategoriaJerarquia[*] c, int[0..1] stock)
+ addCategoriaTipo(String nombre, CategoriaJerarquia superior) : Boolean
+ addCategoriaTipo(String nombre, int[0..1] superior) : Boolean
+ eliminarProducto(Producto p) : Boolean
+ setCentralesRifNroestadoEmpleado(e, String pv) : Boolean
+ setCentralesRifNroestadoEmpleado(e, int[0..1] estado) : Boolean
+ eliminarDescuentoProducto(Producto p) : Boolean
+ setProducto(p) : Producto(int cantidad, int[0..1] estado, double precio, double) : void
+ setCategoriaTipo(int id, String nombre, CategoriaJerarquia superior) : void
+ consultarVisualizaProductoVenta(p) : Producto[*]
+ setPrecioProducto(int productoId, double precio) : Boolean
+ setVentaProducto(int productoId, int[0..1] cantidad) : Boolean
+ consultaEstadisticas(p) : Estadistica
+ setCategoriaTipo(int id, String nombre, int[0..1] Categoria c) : void
+ setPrecioTasa(int tasaId, double precio) : Boolean
+ setNuevoDispensaVenta(int[0..1] tasaId) : Boolean
+ setNuevoDispensaVenta(int[0..1] tasaId, int[0..1] cantidad) : Boolean
+ consultaEstadisticasPorTipo(p, Map<EmpleadoA, TipoPermiso[1..3]>
+ setUbicacionAdmin(int, int[0..1] estado) : Map<ClienteRegistrado, int>
+ setPrecioTasaTotalVentas(int nro, int[0..1] estado) : Map<PrivatizaMoneda, int>
+ setVentaTotalPorVentas(int nro, int[0..1] estado) : Map<PrivatizaVenta, int>
+ setCentralesRif(p) : Boolean
+ setUsuarioCuenta(p) : Integer[0..1]
+ cambiarUsuarioNombreNuevo(p) : Boolean
+ setEstadoCuentaCliente(int cliente, int[0..1] estado) : Boolean
+ setEstadoPrivatizaPorVentas(p, int[0..1] estado) : Boolean
+ crearProducto(Producto Venta[*] p, double precio) : Boolean
Empleado
+ crearTasa(int) : Boolean
+ crearTasaUnPrivatizaMonedaNro(p) : Tasa(int
+ setUbicacionAdmin(int, int[0..1] estado) : Boolean
+ setEstadoPrivatizaPorVentas(p, int[0..1] estado) : Boolean
+ crearProducto(Producto Venta[*] p) : Pedi
+ buscarProducto(p) : Producto(int cantidad, int estado, EstadoFisico, precio: double) : void
+ setEstadoPrivatizaPorVentas(p, int[0..1] estado) : Boolean
+ linkeaPrivatizaTipo(Tipo[*] p) : Boolean
+ buscarProducto(p) : Producto(int cantidad, int estado, EstadoFisico, precio: double) : void
+ setEstadoPrivatizaPorVentas(p, int[0..1] estado) : Boolean
+ setEstadoPrivatizaPorVentas(p, int[0..1] estado) : Pedido[*]
+ setNuevoProducto(Producto p) : Pedido[*]
+ consultarVisualizaProductoVenta(p) : Producto[*]
+ setUbicacionAdmin(int, int[0..1] estado) : Boolean
+ setEstadoCuentaClienteCambiar(int cliente, int[0..1] estado) : Boolean
+ setCodigoCuentaClienteCambiar(int cliente, int[0..1] codigo) : Boolean
+ setEstadoPrivatizaPorVentas(p, int[0..1] estado) : Boolean
+ setNuevoProducto(Producto p) : Pedido[*]
+ setNuevoProducto(Producto p) : Pedido[*]
+ setNuevoProducto(Producto p) : Pedido[*]

1. **Diagrama de Clase**

*Las clases vacías en los diagramas están implementadas en otros diagramas

1.1 Diagrama de Usuarios:

Dividimos los 3 tipos de usuario: Gestor, Empleado y Cliente Registrado, ya que comparten los atributos esenciales (nickname, contraseña, id Único), por ello el Cliente No Registrado va separado.

Respecto a los tipos de permisos de los empleados, hemos creado un <<*enum*>>, así un empleado puede tener más de 1 permiso. Dado que no hemos creado una clase para cada tipo de empleado, todos los métodos de Empleado estarán en dicha clase, en el código nos veremos obligados a comprobar si un empleado tiene el permiso específico para aplicar el método.

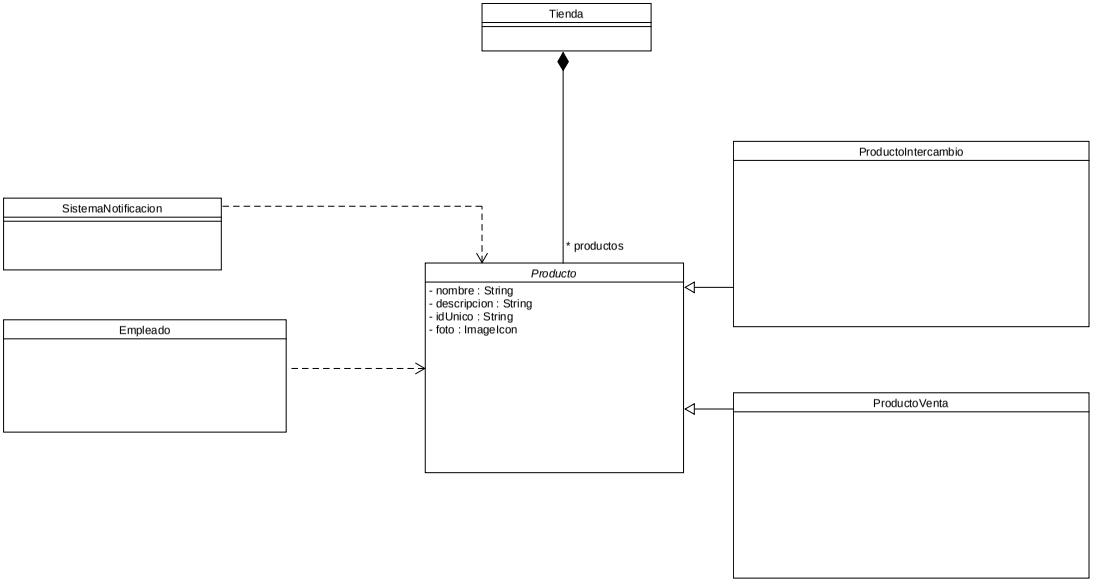
Por otro lado, hemos implementado 2 tipos de servicios:

1. Servicio de Autenticación:
    a. Todo lo necesario para acceder a la aplicación o salirse de ella
2. Sistema de Notificaciones:
    a. Se encarga de notificar a los empleados y a los clientes. Está compuesta por la clase Notificación que indicará el tipo de notificación y su contenido

El gestor tiene acceso a un Historial, donde se almacena todos los datos de las estadísticas (actividad usuarios, recaudación mensual y ventas totales por mes).

Para implementar el filtrado colaborativo, hemos puesto una relación del Cliente Registrado consigo mismo con “similar”, así registramos los clientes que son similares.

Por último, en este diagrama implementamos la clase principal de la aplicación: Tienda, esta clase contiene los métodos necesarios para crear, insertar o borrar productos.

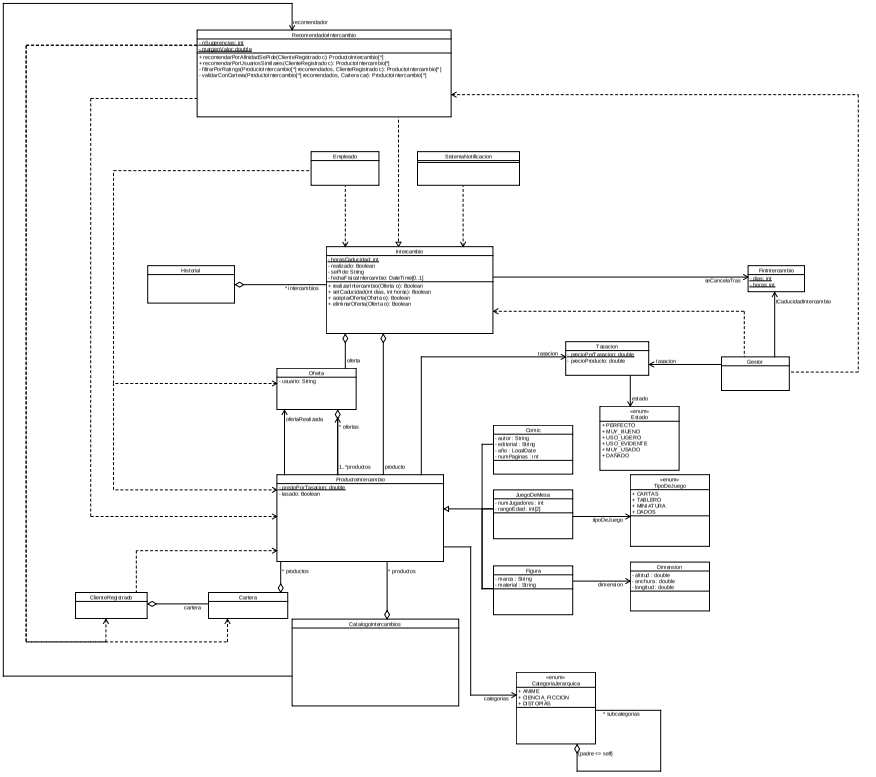

Tienda
* productos
ProductoIntercambio
SistemaNotificacion
Producto
- nombre : String
- descripcion : String
- idUnico : String
- foto : ImageIcon
Empleado
ProductoVenta

1.2 Diagrama de Productos:

En este diagrama definimos la estructura base de los productos mediante la clase **Producto**, que contiene los atributos comunes (nombre, descripción, id único e imagen). Esta clase actúa como superclase abstracta de **ProductoVenta** y **ProductoIntercambio**.

La clase **Tienda** mantiene una composición con Producto, ya que es la encargada de gestionar el conjunto de productos del sistema.

Este diagrama recoge únicamente la estructura general compartida, dejando los comportamientos específicos de venta e intercambio para sus respectivos subdiagramas.


recomendador
Recomendadorintercambio
- infoPreferencias: int
- infoPorcentajeMatch: double
+ recomendarPorAfinidadSeEfectuaClienteRegistrado (c): ProductointercambioE[*]
+ recomendarPorUsuariosSimilares(ClienteRegistrado c): ProductointercambioE[*]
+ filterPorRatio(p:ProductointercambioE[*] recomendados, ClienteRegistrado c): ProductointercambioE[*]
- validarConCartera(ProductointercambioE[*] recomendados, Cartera cart): ProductointercambioE[*]
Empleado
SistemaNotificacion
Intercambio
- fecharealizacion: int
- realizado: Boolean
- fecha: Date
- fechaFinalizarIntercambio: Date(Trim(f), 1)
+ realizarIntercambio(Oferta o): Boolean
+ setCaducidad(int dias, int hora): Boolean
+ aceptarOferta(Oferta o): Boolean
+ eliminarOferta(Oferta o): Boolean
Historial
* Intercambios
aviCancelaTrans
FinIntercambio
- dias: int
- horas: int
CaducidadIntercambio
Transacion
- precioFinalTransacion: double
- precioProducto: double
transacion
transacion
Gestor
estado
«enum»
Estado
+ PERFECTO
+ MUY_BUENO
+ USO_LIGERO
+ USO_EVIDENTE
+ MUY_USADO
+ DAÑADO
Oferta
- usuario: String
oferta
ofertaRealizada
* ofertas
Comic:
- autor : String
- editorial : String
- año : LocalDate
- numPaginas : int
1..*productos
producto
ProductointercambioE
- precioEnTransacion: double
- tasado: Boolean
JuegoDeMesa
- numJugadores : int
- rangoEdad : int[]
tipoDeJuego
«enum»
TipoDeJuego
+ CARTAS
+ TABLERO
+ MINIATURA
+ DADOS
* productos
* productos
Figura
- nrrama : String
- material : String
dimension
Dimension
- althuf : double
- anchura : double
- longitud : double
ClienteRegistrado
cartera
Cartera
CatalogointercambiosE
categorias
«enum»
CategoriaJerarquica
+ ANIME
+ CIENCIA_FICCION
+ DISFORAS
* subcategorias
(padre <> self)

1.2.1 Diagrama de Productos de Intercambio:

En este diagrama desarrollamos el sistema completo de intercambio de productos. El elemento central es la clase Intercambio, que gestiona el proceso entre clientes registrados, incluyendo la caducidad, aceptación de ofertas y la realización final del intercambio.

Cada ProductoIntercambio puede ser tasado previamente mediante la clase Tasacion, donde se fija el precio estimado y el estado físico del producto (modelado mediante el <<*enum*>> Estado). Esto permite valorar correctamente los objetos antes de que entren en el sistema de intercambio.

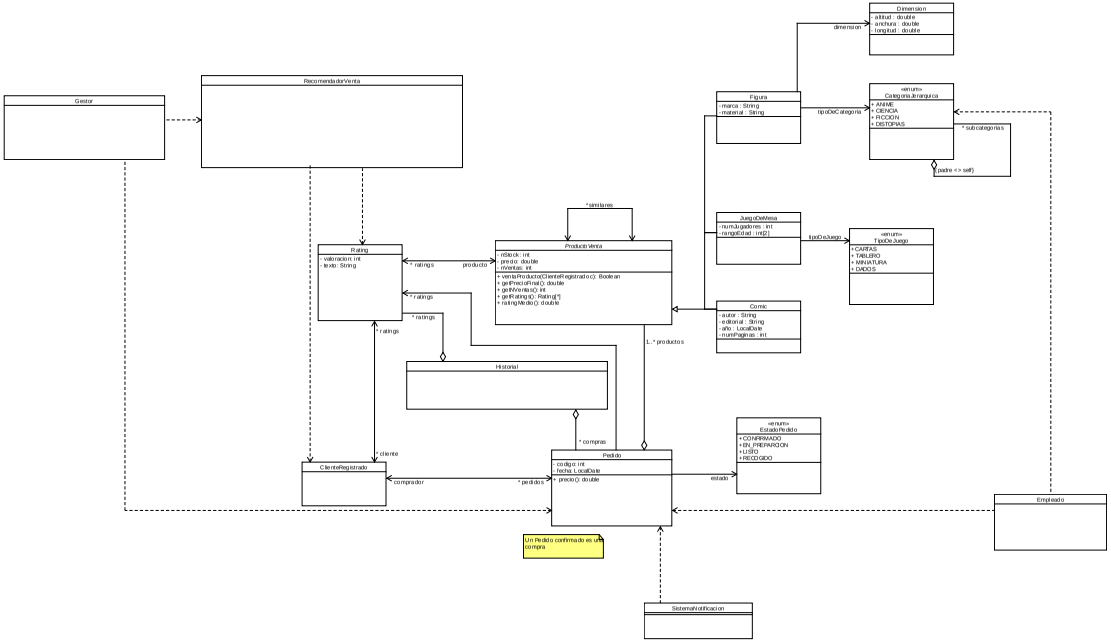
Las Ofertas permiten que otros clientes propongan uno o varios productos a cambio. Un intercambio puede contener múltiples ofertas y productos asociados, reflejando un modelo flexible y realista.

Para gestionar los objetos disponibles, se utiliza un CatalogoIntercambios, mientras que cada cliente dispone de una Cartera, donde almacena los productos que puede ofrecer.

Respecto a la tipología de productos, mantenemos la jerarquía común: Comic, JuegoDeMesa y Figura, cada uno con sus atributos específicos. Se complementa con elementos auxiliares como Dimension, TipoDeJuego y CategoriaJerarquica, permitiendo una clasificación estructurada y extensible.

Además, se implementa un sistema de RecomendadorIntercambio, que sugiere productos en función de afinidad, usuarios similares y validaciones sobre la cartera del cliente, incorporando así filtrado colaborativo en el sistema de intercambios.

Por último, el Gestor puede intervenir en la configuración de tasaciones y categorías, mientras que el SistemaNotificacion informa de eventos relevantes del intercambio.


Gestor
RecomendadorVentas
Rating
- valoracion: int
- texto: String
* ratings
productos
* ratings
* ratings
* ratings
ProductoVenta
+ nStock: int
+ precio: double
+ nVentas: int
+ ventaProducto(ClienteRegistrado c): Boolean
+ getPrecioFinal(): double
+ getNVentas(): int
+ getRating(): Rating[*]
+ ratingMedio(): double
*similares
Figura
- marca : String
- material : String
tipoDeCategoria
Dimension
- altitud : double
- anchura : double
- longitud : double
dimension
«enum»
CategoriaJaponesa
+ ANIME
+ CIENCIA
+ FICCION
+ DISTOPIAS
* subcategorias
{padre <> self}
JuegoDeMesa
+ numJugadores : int
+ rangoEdad : int[2]
tipoDeJuego
«enum»
TipoDeJuego
+ CARTAS
+ TABLERO
+ MINIATURA
+ DADOS
Comic
+ autor : String
+ editorial : String
+ año : LocalDate
+ numPaginas : int
1..* productos
Historial
* compras
Pedido
+ codigo: int
+ fecha: LocalDate
+ precio(): double
estado
«enum»
EstadoPedido
+ CONFIRMADO
+ EN_PREPARACION
+ LISTO
+ RECOGIDO
* cliente
ClienteRegistrado
comprador
* pedidos
Empleado
Un Pedido confirmado es una compra
SistemaNotificacion

El diagrama de clases de los productos de ventas es muy denso y hemos decidido dividirlo en 3 partes:

1.2.2: Diagrama de Ventas 1 (Categorías, Ratings y pedidos):

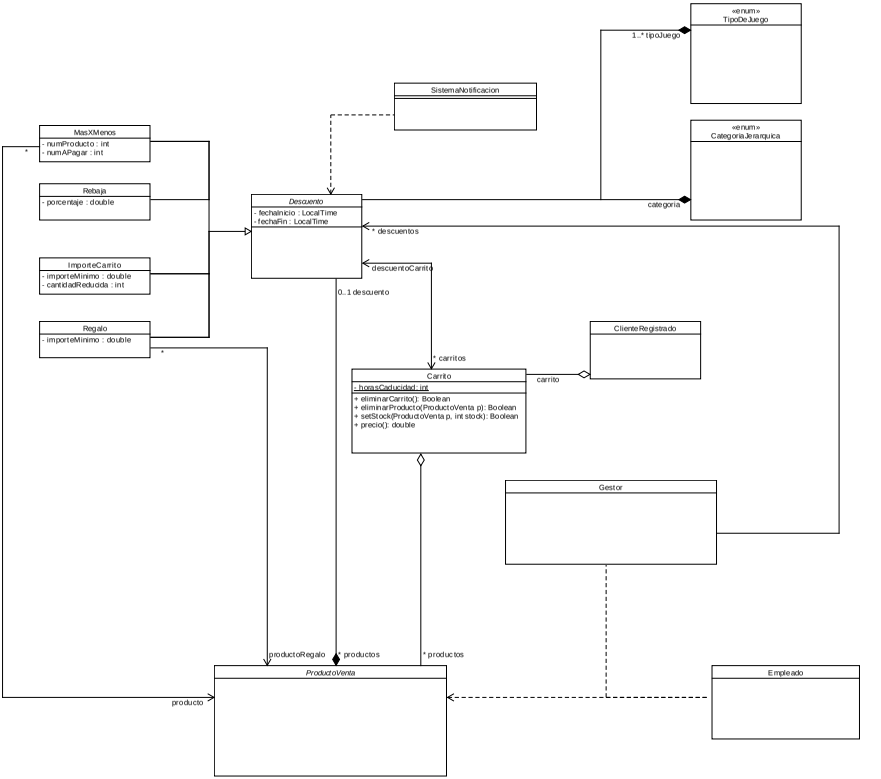
Los productos nuevos tienen ratings (formados por estrellas y texto), puestos por los usuarios que los compran.

Arriba a la izquierda se puede ver el recomendador de productos nuevos, el cual se desarrolla en 1.2.4.

Los pedidos tienen código (para recogerlo), la fecha en la que se ha realizado el pedido y un estado que indica en qué fase está el pedido.

Cada producto nuevo puede ser de tres tipos, y dentro de algunos tipos, tienen ciertos atributos:

- Comic: no tiene atributos especiales.
- Juegos de mesa: se indica el tipo de juego.
- Figuras: Tiene dimensión y categorías jerárquicas, es decir, que pueden tener padre y subcategorías.

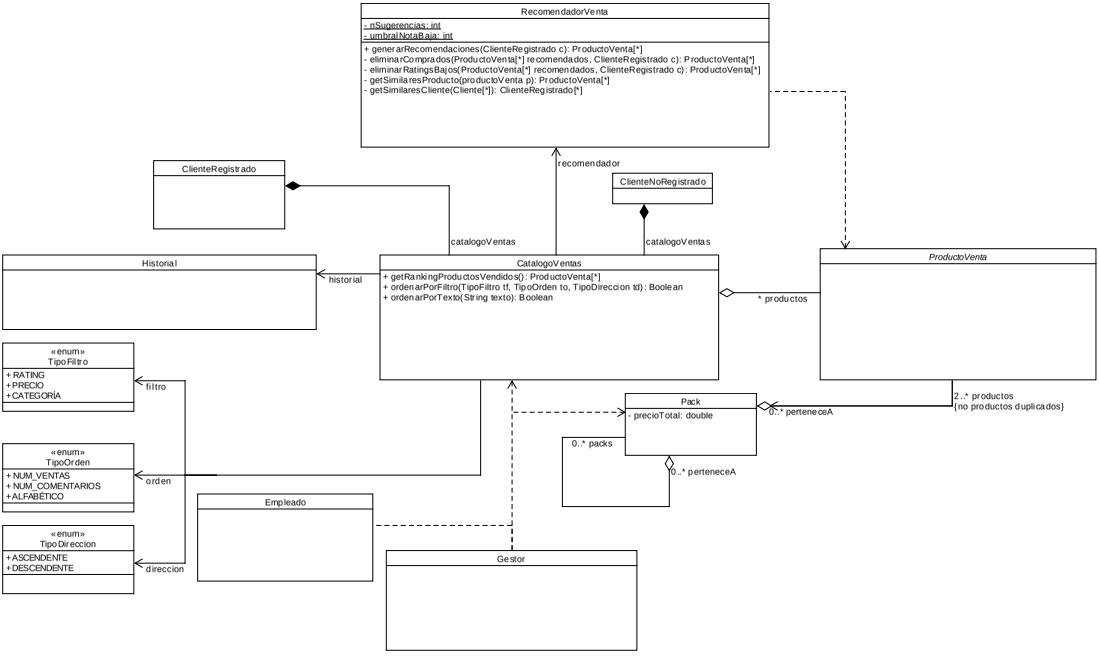

«enum»
TipoDeJuego
1..* tipoJuego
SistemaNotificacion
«enum»
CategoriaJerarquica
MasXMenos
- numProducto : int
- numAPagar : int
*
Rebaja
- porcentaje : double
Descuento
- fechaInicio : LocalTime
- fechaFin : LocalTime
categoria
* descuentos
ImporteCarrito
- importeMinimo : double
- cantidadReducida : int
descuentoCarrito
0..1 descuento
Regalo
- importeMinimo : double
*
* carritos
ClienteRegistrado
Carrito
- horasCaducidad : int
+ eliminarCarrito(): Boolean
+ eliminarProducto(ProductoVenta p): Boolean
+ setStock(ProductoVenta p, int stock): Boolean
+ precio(): double
carrito
Gestor
productoRegalo
productos
* productos
ProductoVenta
producto
Empleado

1.2.3: Diagrama de Ventas 2 (Carrito y descuentos):

Cada cliente tiene un carrito, que está formado por productos nuevos y tiene una fecha de caducidad definida por el gestor, tras la cual, el carrito suelta los objetos.

Los descuentos pueden afectar a un objeto o a categorías y pueden ser de 4 tipos:

- MasXMenos: el cliente paga menos productos de los que se lleva. Es de un producto en concreto. Ejemplo: 3x2.
- Rebaja: Se hace un descuento en porcentaje respecto del total del carrito.
- Importe Carrito: al llegar a cierta cantidad, te rebajan una cantidad de dinero.
- Regalo: al llegar a cierta cantidad, te regalan un producto.

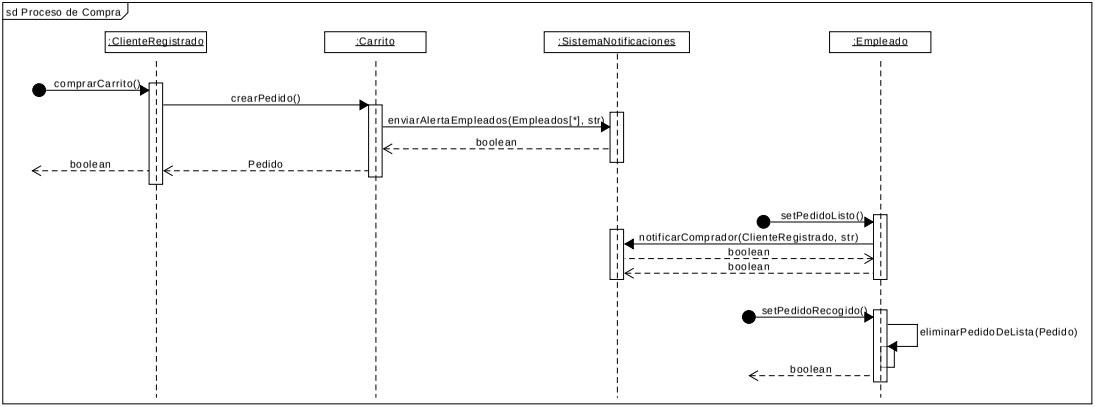

RecomendadorVenta
- nSugerencias: int
- umbralNotaBaja: int
+ generarRecomendaciones(ClienteRegistrado c): ProductoVenta[*]
- eliminarComprados(ProductoVenta[*] recomendados, ClienteRegistrado c): ProductoVenta[*]
- eliminarRatingsBajos(ProductoVenta[*] recomendados, ClienteRegistrado c): ProductoVenta[*]
- getSimilaresProducto(productoVenta p): ProductoVenta[*]
- getSimilaresCliente(Cliente[*]): ClienteRegistrado[*]
recomendador
ClienteRegistrado
ClienteNoRegistrado
catalogoVentas
catalogoVentas
Historial
historial
CatalogoVentas
+ getRankingProductosVendidos(): ProductoVenta[*]
+ ordenarPorFiltro(TipoFiltro tf, TipoOrden to, TipoDireccion td): Boolean
+ ordenarPorTexto(String texto): Boolean
* productos
ProductoVenta
«enum»
TipoFiltro
+ RATING
+ PRECIO
+ CATEGORÍA
filtro
Pack
- precioTotal: double
0..* perteneceA
2..* productos
{no productos duplicados}
0..* packs
0..* perteneceA
«enum»
TipoOrden
+ NUM_VENTAS
+ NUM_COMENTARIOS
+ ALFABÉTICO
orden
Empleado
«enum»
TipoDireccion
+ ASCENDENTE
+ DESCENDENTE
direccion
Gestor

1.2.4: Diagrama de Ventas 3 (Catálogo, recomendador, packs y búsquedas):

El cliente tiene acceso al catálogo (para ver todos los productos que hay a la venta), el cual se ordena de cierta manera según el recomendador (no se recomiendan comprados ni productos valorados de mala manera por el usuario y se recomiendan más los productos similares a los que ha comprado el cliente y los que han comprado clientes similares a él).

Este catálogo tiene un sistema de búsqueda por filtros (rating, precio y categorías jerárquicas), que se pueden ordenar según el número de ventas, el número de comentarios o en orden alfabético de forma ascendente o descendente.

Tanto el gestor como los empleados con permiso de productos pueden crear packs que están formados por otros productos y packs y tienen un precio menor que el total de los productos que los conforman.


sd Proceso de Compra
:ClienteRegistrado
:Carrito
:SistemaNotificaciones
:Empleado
comprarCarrito()
crearPedido()
enviarAlertaEmpleados(Empleados[*], str)
boolean
boolean
Pedido
setPedidoListo()
notificarComprador(ClienteRegistrado, str)
boolean
boolean
setPedidoRecogido()
eliminarPedidoDeLista(Pedido)
boolean

2. Diagramas de Secuencia

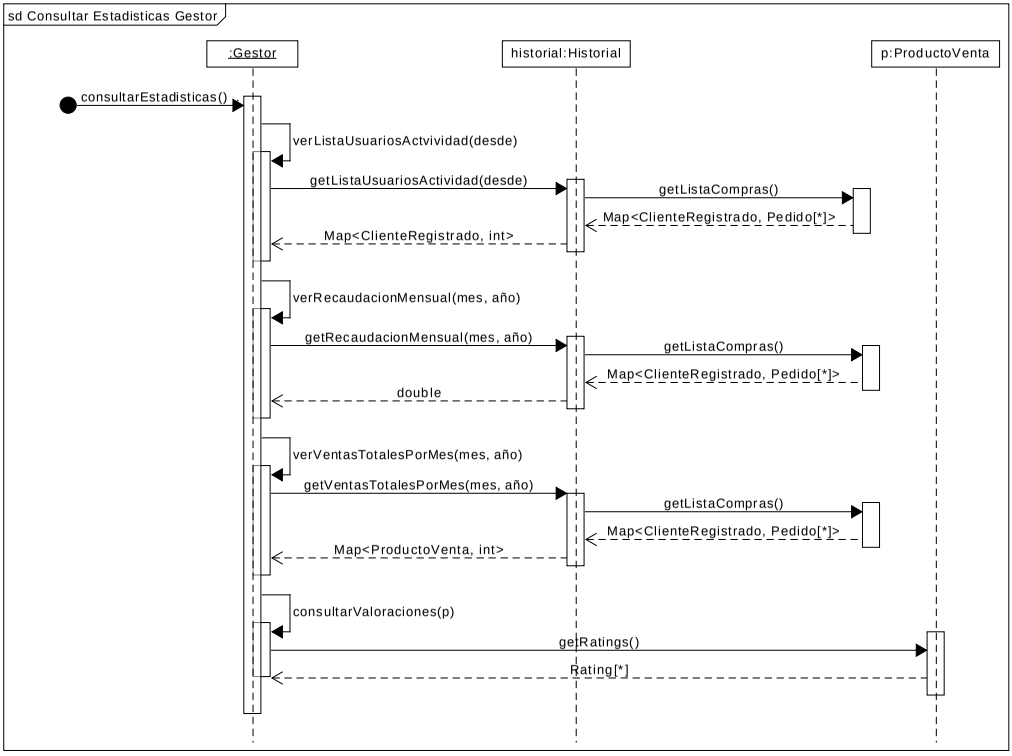
2.1. Proceso de compra:

El proceso de compra en nuestra app se divide en tres fases realizadas por las siguientes clases: el Cliente Registrado, el Carrito, el Sistema de Notificaciones y el Empleado.

La secuencia comienza nada más llamar a comprarCarrito() tras pagar el mismo. El carrito ejecutara crearPedido() y el sistemaDeNotificaciones enviara una alerta a los empleados. Este último retornara un true si todo ha ido bien. Consecutivamente se retornará el objeto Pedido y finalmente otro boolean que es el que confirma que todo ha ido bien.

Continua el proceso cuando se pone el pedido en “listo” enviando una notificación al cliente para que lo vaya a recoger. Retornando un primer true para confirmar que se ha enviado la notificación y otro para confirmar que el pedido está en listo.

Por último, el pedido se actualiza en recogido en cuanto el empleado lo confirma llamando así a eliminarPedidoDeLista() y confirmando otra vez que este subproceso ha ido bien con otra secuencia de dos trues como retorno.


sd Consultar Estadisticas Gestor
:Gestor
historial:Historial
p:ProductoVenta
consultarEstadisticas()
verListaUsuariosActvividad(desde)
getListaUsuariosActividad(desde)
getListaCompras()
Map<ClienteRegistrado, Pedido[*]>
Map<ClienteRegistrado, int>
verRecaudacionMensual(mes, año)
getRecaudacionMensual(mes, año)
getListaCompras()
Map<ClienteRegistrado, Pedido[*]>
double
verVentasTotalesPorMes(mes, año)
getVentasTotalesPorMes(mes, año)
getListaCompras()
Map<ClienteRegistrado, Pedido[*]>
Map<ProductoVenta, int>
consultarValoraciones(p)
getRatings()
Rating[*]

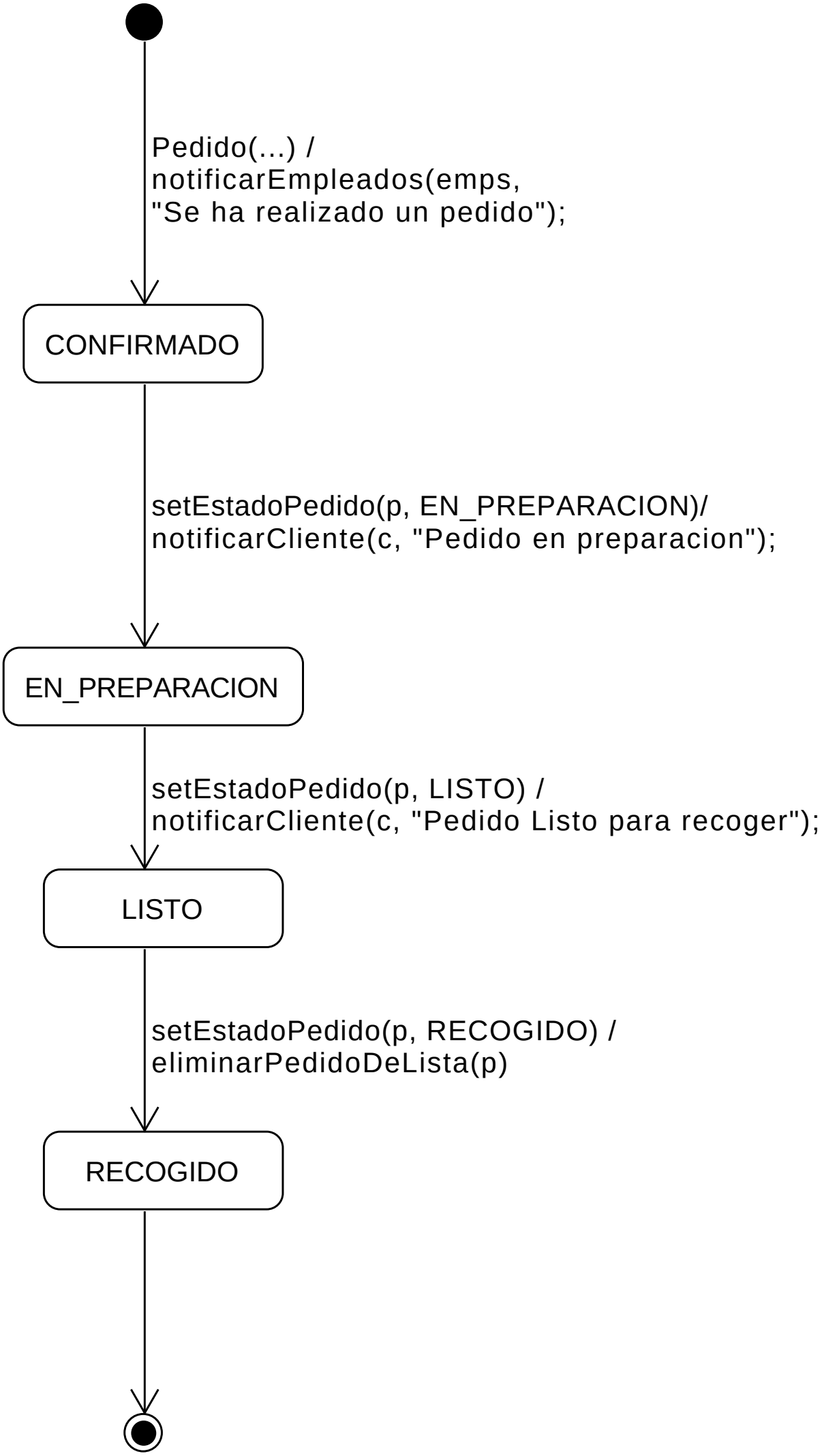
2.2. Consulta de Estadísticas del Gestor:

El proceso de consulta de estadísticas en nuestra app se divide en varias fases realizadas por las siguientes clases: el Gestor, el Historial y el ProductoVenta.

La secuencia comienza al llamar a consultarEstadisticas(). El gestor ejecutará internamente verListaUsuariosActvividad(desde), lo que hará que este solicite al historial la actividad mediante el método getListaUsuariosActividad(desde). El historial, se llamará a su propio método getListaCompras(). Este último retornará un Map<ClienteRegistrado, Pedido[*]>, y el historial retornará al gestor un Map<ClienteRegistrado, int> con la actividad de los usuarios.

Continúa el proceso cuando el gestor ejecuta sus métodos internos verRecaudacionMensual(mes, año) y verVentasTotalesPorMes(mes, año). En ambos casos, el gestor consulta al historial mediante getRecaudacionMensual(mes, año) y getVentasTotalesPorMes(mes, año) respectivamente. En ambas situaciones, el historial vuelve a obtener los datos necesarios invocando getListaCompras(). Se retornará un valor tipo double para la recaudación y un Map<ProductoVenta, int> para las ventas totales.

Por último, el gestor realiza una consulta de valoraciones interna llamada consultarValoraciones(p), que desencadena la llamada a getRatings() directamente al objeto ProductoVenta. Este retornará una lista con las valoraciones (Rating[*]).


Pedido(...) /
notificarEmpleados(emps,
"Se ha realizado un pedido");
CONFIRMADO
setEstadoPedido(p, EN_PREPARACION)/
notificarCliente(c, "Pedido en preparacion");
EN_PREPARACION
setEstadoPedido(p, LISTO) /
notificarCliente(c, "Pedido Listo para recoger");
LISTO
setEstadoPedido(p, RECOGIDO) /
eliminarPedidoDeLista(p)
RECOGIDO

3. Diagramas de Transición de Estados

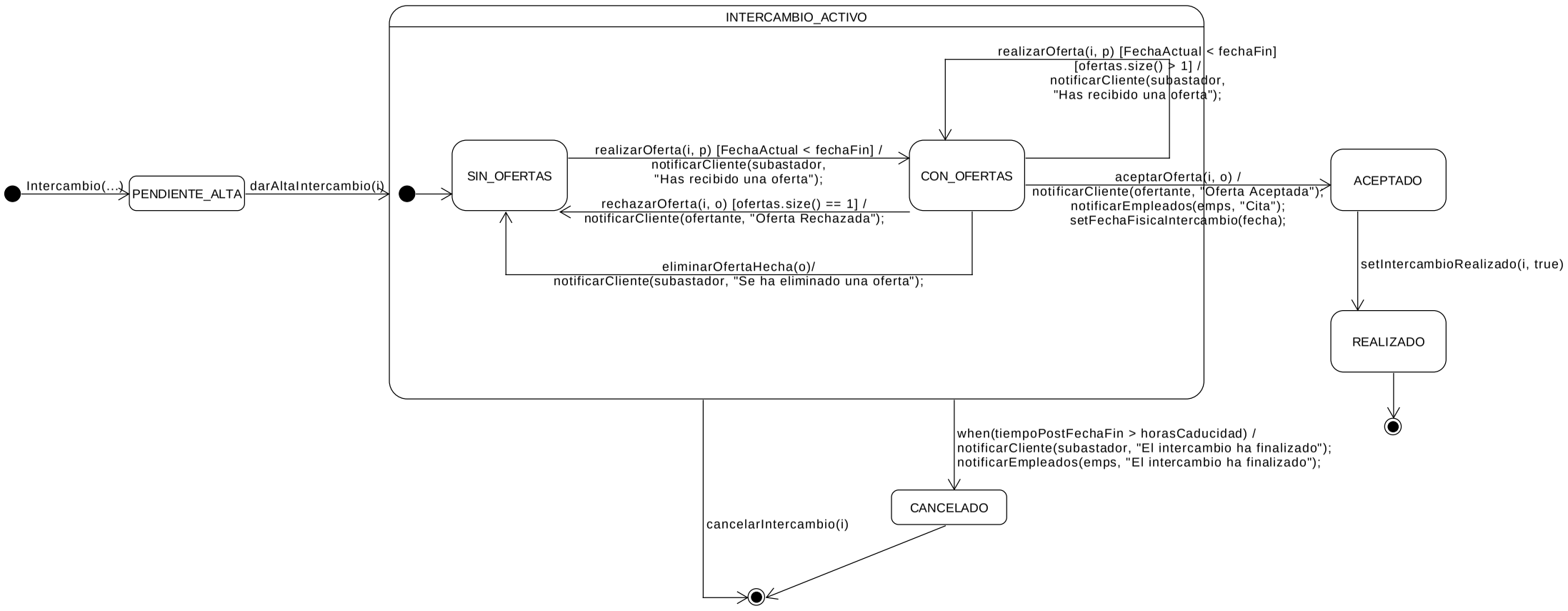
En este apartado se adjuntarán los diagramas de transición de estados, hemos elegido hacerlo de las clases Pedido y Intercambio, al ser clases troncales en el diseño de esta aplicación:

3.1 Ciclo de vida de un Pedido

Un objeto de tipo Pedido nace después del pago confirmado del carrito de un usuario, nace con la llamada al constructor. Nada más se creado el pedido, se notifica a los empleados con permisos de pedidos, “emps”. El pedido está confirmado, el primer estado en el diagrama de transición de estados.

Se puede observar claramente la linealidad en los estados de la clase Pedido. El cambio en el estado del pedido, que son una enumeración (CONFIRMADO, EN_PREPARACION, LISTO, RECOGIDO), se hacen a través de un método setter setEstadoPedido(Pedido p, EstadoPedido e), por un empleado con permisos de pedidos. Con cada cambio de estado, se notifica al cliente un mensaje y la información del pedido.

En el último cambio de estado desde LISTO a RECOGIDO, hay un paso el cual no está especificado en el diagrama ya que es una comprobación física del empleado. El empleado revisa el DNI físico que trae el cliente y su número de pedido, que coincidan con los del pedido en el sistema, simplificando el software y delegando la reponsabilidad en el empleado con permisos de pedido.


INTERCAMBIO_ACTIVO
Intercambio(...)
PENDIENTE_ALTA
darAltaIntercambio(i)
SIN_OFERTAS
realizarOferta(i, p) [FechaActual < fechaFin] /
notificarCliente(subastador,
"Has recibido una oferta");
CON_OFERTAS
realizarOferta(i, p) [FechaActual < fechaFin]
[ofertas.size() > 1] /
notificarCliente(subastador,
"Has recibido una oferta");
rechazarOferta(i, o) [ofertas.size() == 1] /
notificarCliente(ofertante, "Oferta Rechazada");
aceptarOferta(i, o) /
notificarCliente(ofertante, "Oferta Aceptada");
notificarEmpleados(emps, "Cita");
setFechaFisicaIntercambio(fecha);
ACEPTADO
setIntercambioRealizado(i, true)
REALIZADO
eliminarOfertaHecha(o)/
notificarCliente(subastador, "Se ha eliminado una oferta");
when(tiempoPostFechaFin > horasCaducidad) /
notificarCliente(subastador, "El intercambio ha finalizado");
notificarEmpleados(emps, "El intercambio ha finalizado");
CANCELADO
cancelarIntercambio(i)

3.2 Ciclo de vida de un Intercambio

El objeto de tipo intercambio nace en PENDIENTE_ALTA, y pasa a estar ACTIVO cuando lo da de alta un empleado con permisos de pedidos, para revisar que el intercambio tenga los datos correctos.

Para hacer este diagrama de transición hemos utilizado un estado compuesto llamado INTERCAMBIO_ACTIVO para encapsular los intercambios que siguen en activo. Este estado permite que un empleado con permisos de intercambios pueda manualmente cancelar o que caduque un intercambio. Esto ocurrirá independientemente de en qué estado este en la negociación.

Para realizar una oferta se revisa que la fecha actual es menor que la fecha fin, porque si no el intercambio estaría cerrado, no se permitiría a ningún usuario hacer una oferta, pero el intercambiador sigue pudiendo aceptar o rechazar ofertas. La caducidad es un periodo tiempo que se le deja al usuario que ha iniciado el intercambio para poder seguir aceptando y rechazando ofertas. Una vez transcurre este periodo el intercambio pasa a estar cancelado y el usuario no podrá aceptar ni rechazar más ofertas.

Una vez que se acepta una oferta el intercambio pasa a estar en estado ACEPTADO, y deja de ser un intercambio activo lo que no permite al usuario acetar ni rechazar más ofertas, y la caducidad y el tiempo de caducidad deja de afectar al intercambio, previo a pasar a este estado se pone una fecha para el intercambio físico.

El sistema de notificaciones envía notificaciones para comunicar a los usuarios involucrados después de algunos cambios de estados. Por ejemplo, cuando se realiza una oferta se notifica al usuario que la recibe o cuando se acepta una oferta se notifica al ofertante. También se notifica a los empleados cuando se acepta el intercambio con la fecha del intercambio para que estén presentes.